

VISUALIZATION

Data visualization is the process of showing data in a graphical or visual form so it becomes easier to understand and analyze. Instead of reading raw numbers, we use charts, graphs, and plots to quickly identify patterns, trends, and insights.

In Python, there are several powerful libraries for data visualization such as Matplotlib, Seaborn, and Plotly.

Now, let's explore Matplotlib and Seaborn in more detail.

MATPLOTLIB

Matplotlib, introduced in 2002 by John Hunter. It is for plotting 2D plots which is built on numpy. A plot of matplotlib contains:

1 Figure

2 Axes

3 Axis

4 Artists

Figure

The figure is the overall window or canvas where all plots are drawn. A figure can contain one or multiple plots.

Axes

Axes are the plots, and they are artist attached to the figure. A plot can have 2 or 3 axes. `set_xlabel()`, `set_ylabel()` are the functions used to set the labels of x and y respectively.

Axis

It is responsible for generating ticks or the limits on the axes.

Artists

The whatever content visible in a figure are the artists.

PYPLOT

Pyplot is a sub library of matplotlib where all the utilities lie under. It has different types of plots including bar graphs, scatter plots, pie charts, histograms, area charts.

We import pyplot from matplotlib as following:

```
1 import matplotlib.pyplot as plt
```

We also import numpy as a support for the matplotlib :

```
import numpy as np
```

Some of the plots in matplotlib:

LINE CHART

A line chart is a graph that displays data points connected by straight lines to show trends over time.

The default `plot()` is used for line charts.

Code snippet:

```
import matplotlib.pyplot as plt

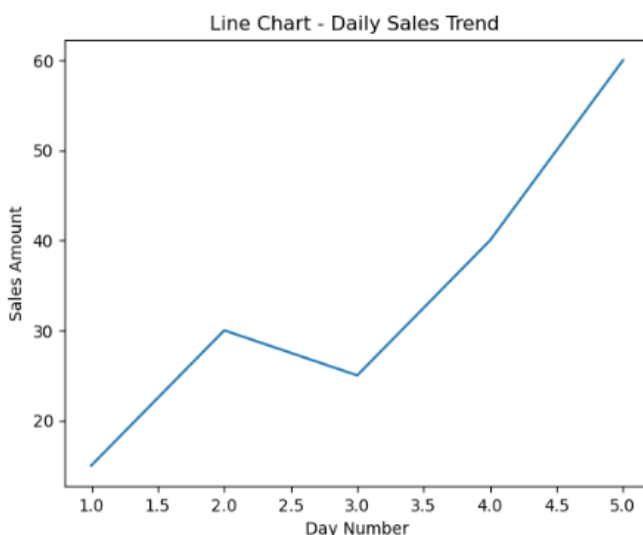
days = [1, 2, 3, 4, 5]
sales = [15, 30, 25, 40, 60]

plt.plot(days, sales)

plt.title("Line Chart - Daily Sales Trend ")
plt.xlabel("Day Number")
plt.ylabel("Sales Amount")

plt.show()
```

Output:



Description:

The line chart shows the sales trend over different months. The graph indicates that sales gradually increase over time, with some months showing higher growth compared to others. By connecting data points with lines, the chart clearly highlights the upward and downward movement in sales performance across the months.

BAR GRAPH

Rectangular bars are used in bar chart as the height represent the frequency of a particular element. `bar()` is used for plotting bar graphs, it has parameters like x, y, width, color.

Code snippet:

```
import matplotlib.pyplot as plt

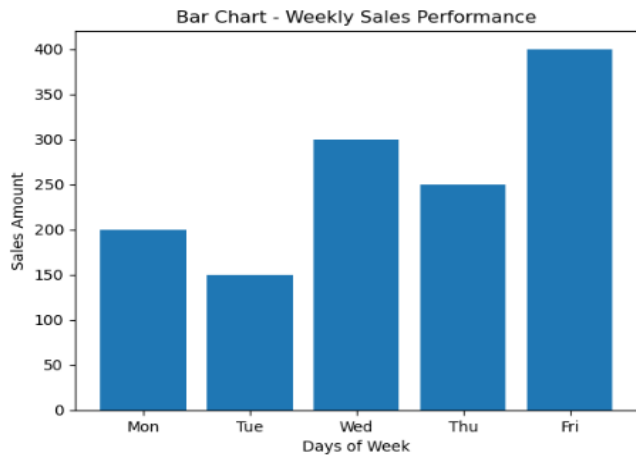
days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri']
sales = [200, 150, 300, 250, 400]

plt.bar(days, sales)

plt.title("Bar Chart - Weekly Sales Performance")
plt.xlabel("Days of Week")
plt.ylabel("Sales Amount")

plt.show()
```

Output:



Description:

The bar graph compares sales values for different months using vertical bars. The height of each bar represents the amount of sales in that month. From the graph, it can be observed that some months recorded higher sales while others had lower performance, making comparison between categories simple and effective.

HISTOGRAM

Histogram is a type of bar graph where the graph is represented in groups. hist() is used for plotting histogram with parameters x, bins, color, edgecolor.

Code snippet:

```
import matplotlib.pyplot as plt

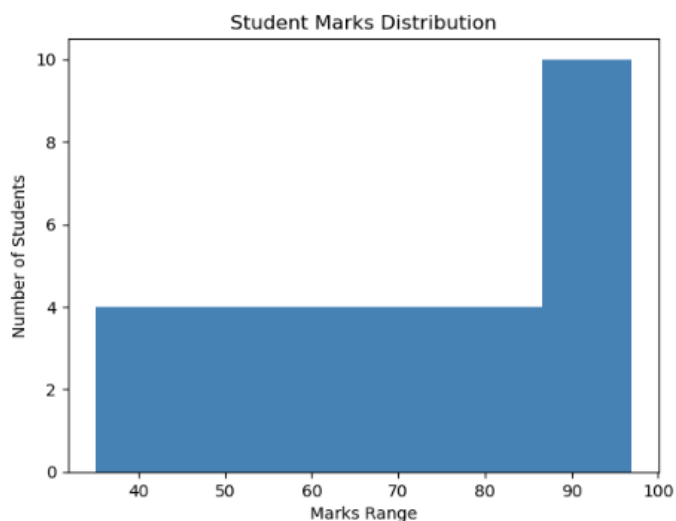
marks = [35, 40, 42, 45, 47, 50, 52, 55, 58, 60, 62, 65, 67, 70, 72, 75, 78, 80, 82, 85, 87, 88, 90, 91, 92, 93, 94, 95, 96, 97]

plt.hist(marks, bins=6, color='steelBlue')

plt.title("Student Marks Distribution")
plt.xlabel("Marks Range")
plt.ylabel("Number of Students")

plt.show()
```

Output:



Description:

The histogram represents the frequency distribution of numerical data. The graph groups data into intervals (bins) and shows how frequently values occur within each range. The graph indicates that some ranges contain more data points than others, helping to understand the concentration and spread of the dataset.

SCATTER PLOT

Scatter plot uses dots to depict the relationship between the data. `scatter()` is used for plotting scatter plot and scatter plot can be done for multiple datasets at the same time by differentiating it with colors. Legends help to achieve this difference.

Code snippet:

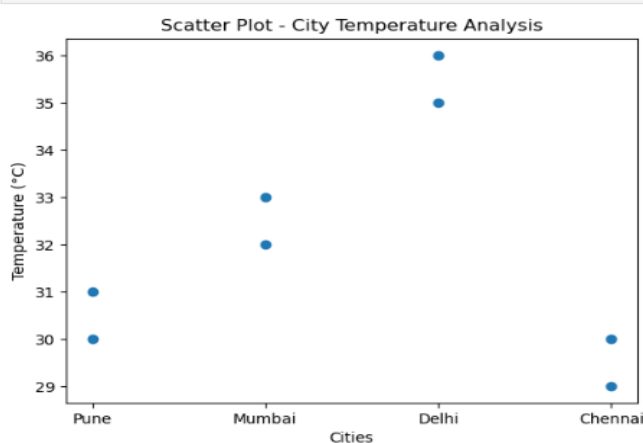
```
import matplotlib.pyplot as plt

cities = ['Pune', 'Mumbai', 'Delhi', 'Pune', 'Mumbai', 'Delhi', 'Chennai', 'Chennai']
temperature = [30, 32, 35, 31, 33, 36, 29, 30]

plt.scatter(cities, temperature)

plt.title("Scatter Plot - City Temperature Analysis")
plt.xlabel("Cities")
plt.ylabel("Temperature (°C)")
plt.show()
```

Output:



Description:

The scatter plot displays the relationship between two variables using data points. The graph shows how one variable changes with respect to another and helps identify trends, clusters, or correlations. From the graph, it can be observed that the points follow a certain pattern, indicating a relationship between the variables.

PIECHART

Pie charts are used to plot data of same kind which means the same series of data where the different elements are divide based on their percentage.

Code snippet:

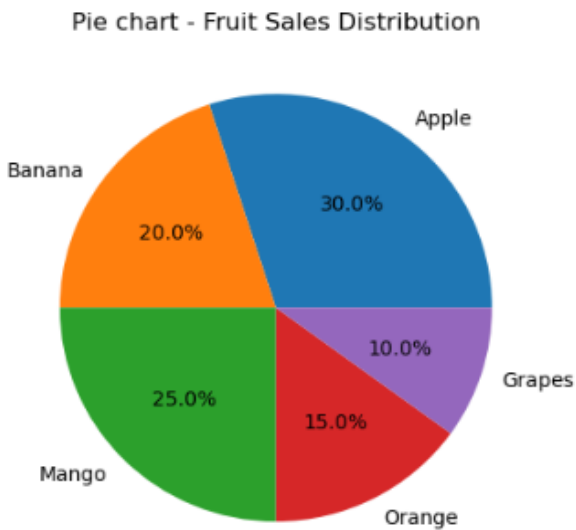
```
import matplotlib.pyplot as plt

fruits = ['Apple', 'Banana', 'Mango', 'Orange', 'Grapes']
sales = [30, 20, 25, 15, 10]

plt.pie(sales, labels=fruits, autopct='%1.1f%%')

plt.title("Pie chart - Fruit Sales Distribution")
plt.show()
```

Output:



Description:

The pie chart represents the distribution of fruit sales among different fruit categories. Each slice of the chart shows the percentage contribution of a fruit to the total sales. The graph indicates that some fruits contribute a larger share of sales, while others contribute a smaller portion, making it easy to compare overall sales distribution visually.

SEABORN

Seaborn is a visualization library in python which is built on top of matplotlib. It is advanced than matplotlib and have different features which are default like style, color palette. Seaborn has different categories of plots like relational plot, categorical plot, distribution plot, regression plot, matrix plot.

Let us see the different plots in each category.

1 Relational Plots:

- scatterplot()
- lineplot()
- relplot()

2 Categorical Plots:

- barplot()
- countplot()
- boxplot()
- violinplot()
- swarmplot()
- pointplot()
- catplot()

3 Distribution Plots:

- histplot()
- kdeplot()
- rugplot()
- distplot()

4 Relational Plots:

- regplot()
- lmpplot()

5 Matrix Plots:

- heatmap()
- clustermap()

Here are some sample codes for some of the graphs.

LINE PLOT:

A Seaborn line plot is used to display trends or relationships between two variables by connecting data points with a line in an attractive statistical format.

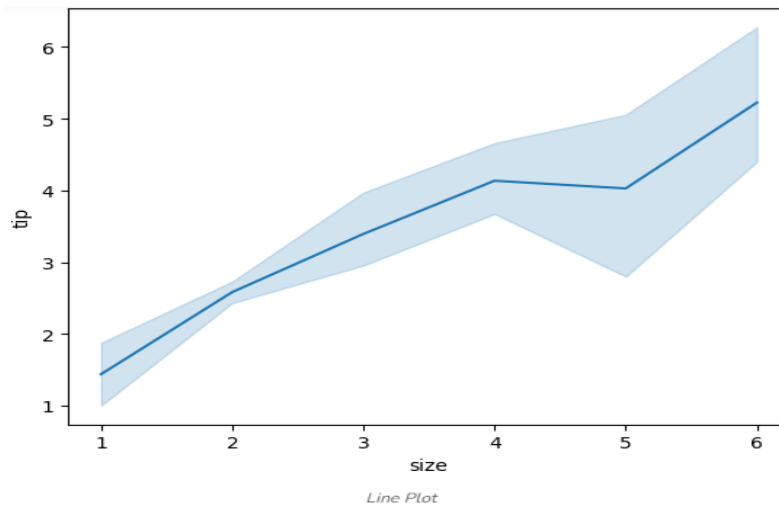
Code Snippet:

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.lineplot(data=tips, x="size", y="tip")

plt.show()
```

Output:



Description:

The Seaborn line plot shows the relationship between the size of customer groups and the tip amount. The graph indicates that as the group size changes, the tip amount also varies. The smooth line representation helps in understanding the overall trend and behavior of the data.

REL PLOT:

A Seaborn relplot is used to visualize the relationship between two variables using different relational plots like scatter plots or line plots.

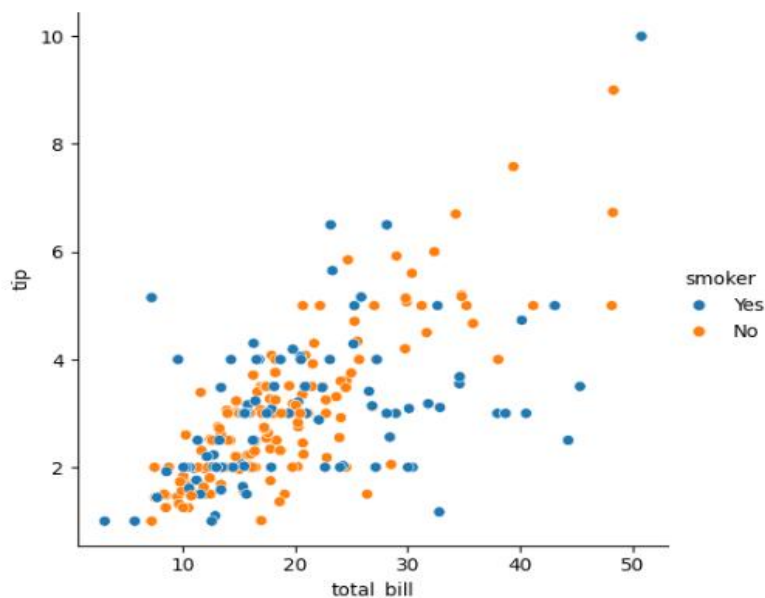
Code Snippet:

```
import seaborn as sns
import matplotlib.pyplot as plt
sns.load_dataset("tips")

sns.relplot(data=tips, x="total_bill", y="tip", hue="smoker")

plt.show()
```

Output:



Description:

The relational plot shows the relationship between total bill amount and tip amount. Different colors are used to represent smoker and non-smoker categories. The graph indicates that higher total bills generally correspond to higher tip amounts, while also allowing comparison between the two categories.

BOXPLOT

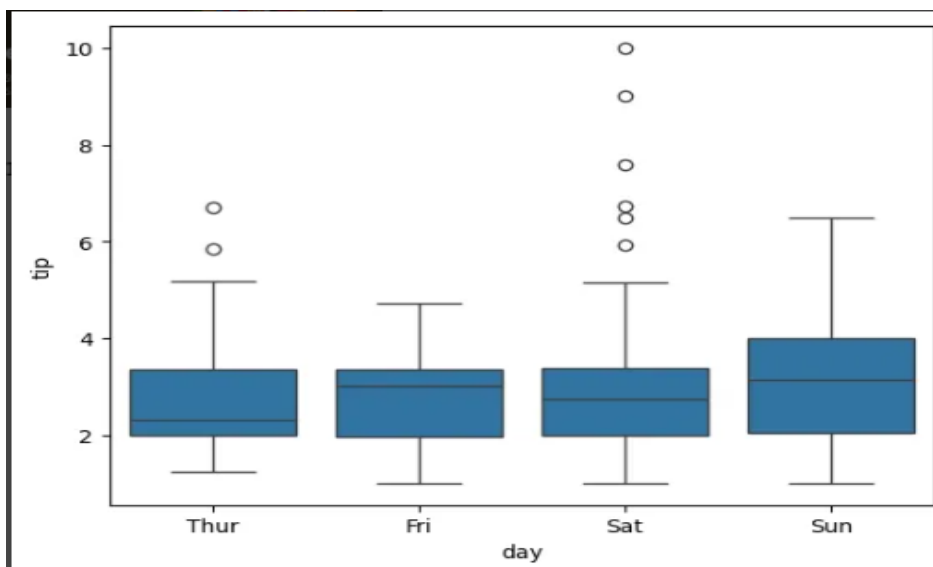
A box plot is used to display the distribution of numerical data through quartiles. It helps in identifying median, spread of data, and outliers.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.boxplot(data=tips, x="day", y="total_bill")

plt.show()
```

Output:



Description:

The box plot represents the distribution of total bill values across different days. The graph shows the median, spread, and possible outliers in the data. It indicates variations in billing patterns between days and helps identify days with higher or more spread-out bill values.

VIOLINPLOT

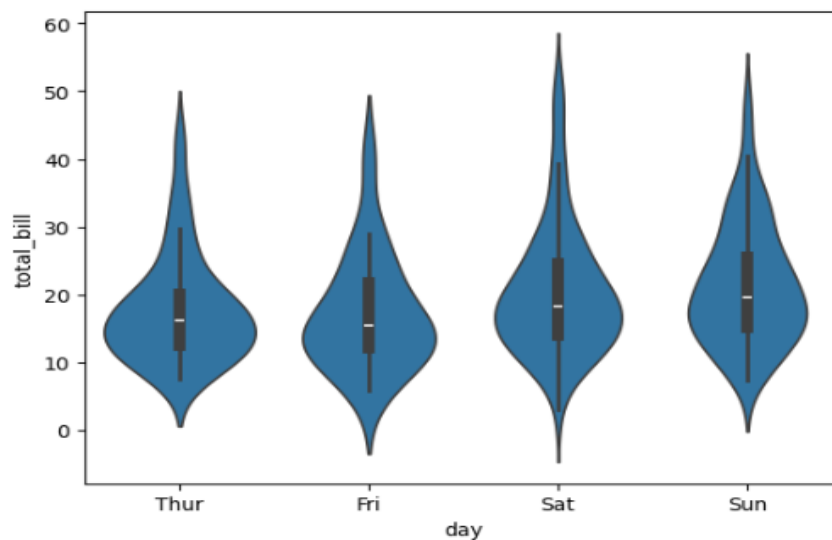
A violin plot is used to show the distribution and density of numerical data across different categories. It combines features of a box plot and a density plot.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.violinplot(data=tips, x="day", y="total_bill")

plt.show()
```


Output:



Description:

The violin plot displays the distribution and density of total bill values for different days. The graph indicates where the data is more concentrated through wider sections and where fewer values occur through narrower sections. It provides a detailed view of data distribution across categories..

KDE PLOT

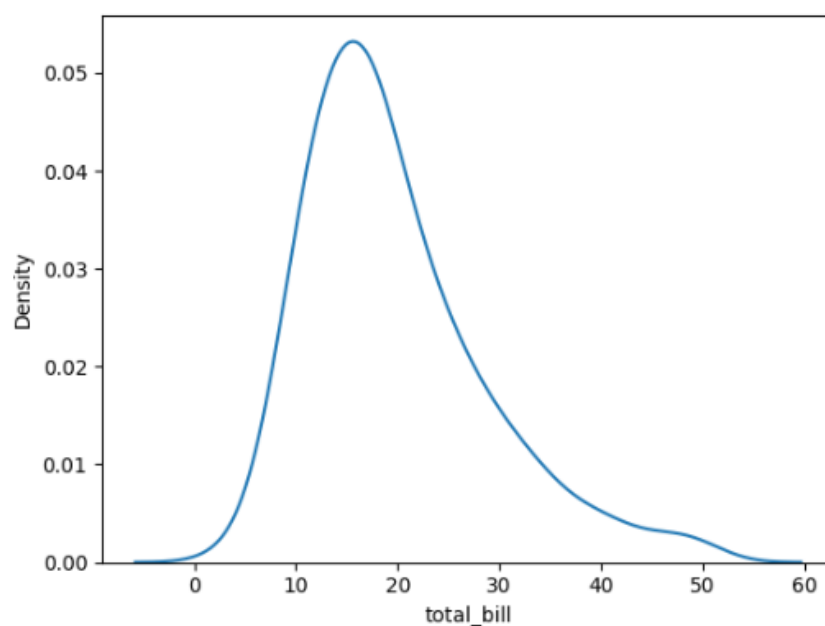
KDE stands for kernel density estimate. It creates a curve as based on probability. It creates single graph for multiple data samples.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.kdeplot(data=tips, x="total_bill")

plt.show()
```

Output:



Description:

The KDE plot shows the probability density distribution of the dataset using a smooth curve. The graph indicates where the data values are concentrated and how the distribution spreads across the range. Peaks in the curve represent areas with higher data frequency.

DISTPLOT

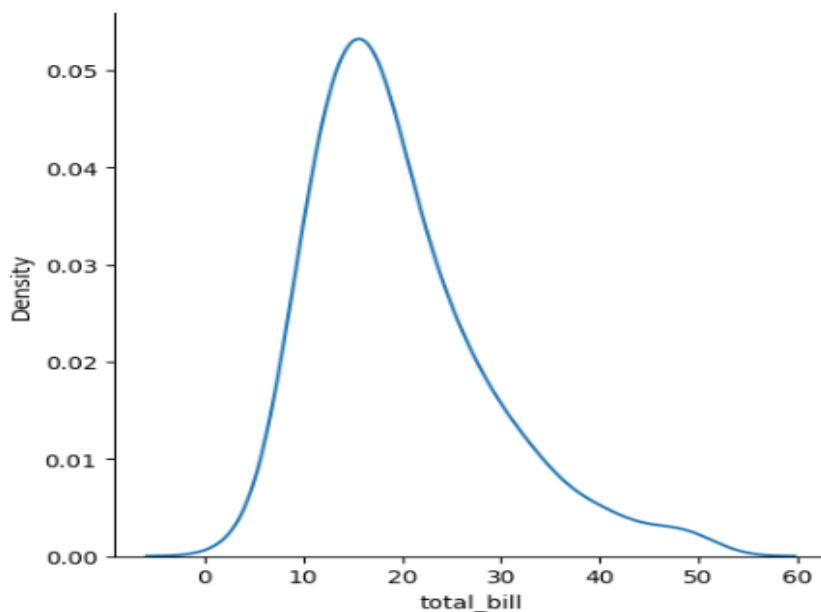
A distplot is used to show the distribution of numerical data using a histogram and density curve together. It helps in understanding the overall spread and pattern of data.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.displot(data=tips, x="total_bill", kind="kde")

plt.show()
```

Output:



Description:

The distplot combines a histogram and density curve to represent the distribution of numerical data. The graph indicates the overall spread, central tendency, and frequency pattern of the dataset, helping in understanding how the values are distributed.

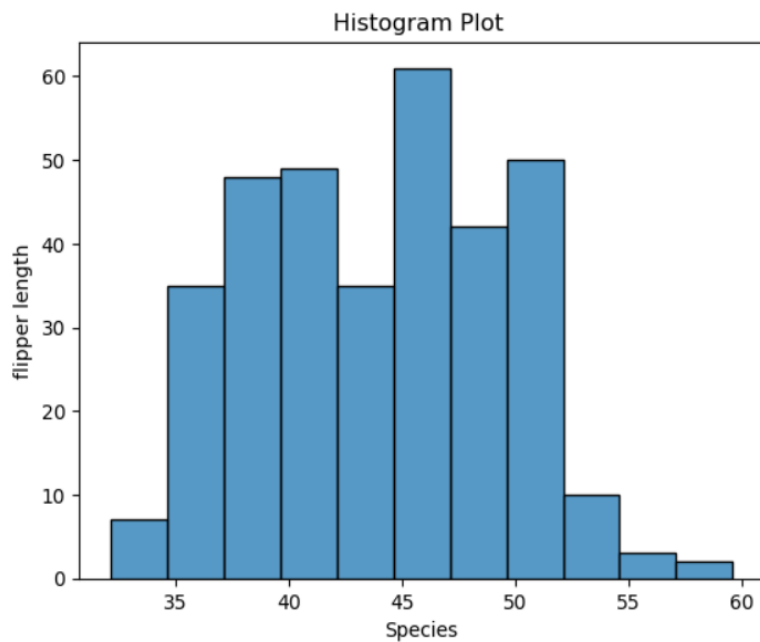
HISTPLOT

A histplot is used to display the frequency distribution of data by dividing values into intervals called bins. It helps in analyzing how data is distributed across different ranges.

Code Snippet:

```
1 import matplotlib.pyplot as plt
2 import pandas as pd
3 import seaborn as sns
4
5 sns.histplot(x='bill_length_mm', data=df, palette="Set2")
6 plt.title('Histogram Plot ')
7 plt.ylabel('flipper length')
8 plt.xlabel('Species')
9 plt.show()
```

Output:



Description:

The histplot represents the frequency distribution of numerical data using bins. The graph indicates which value ranges contain more observations and helps analyze the overall distribution and spread of the dataset.

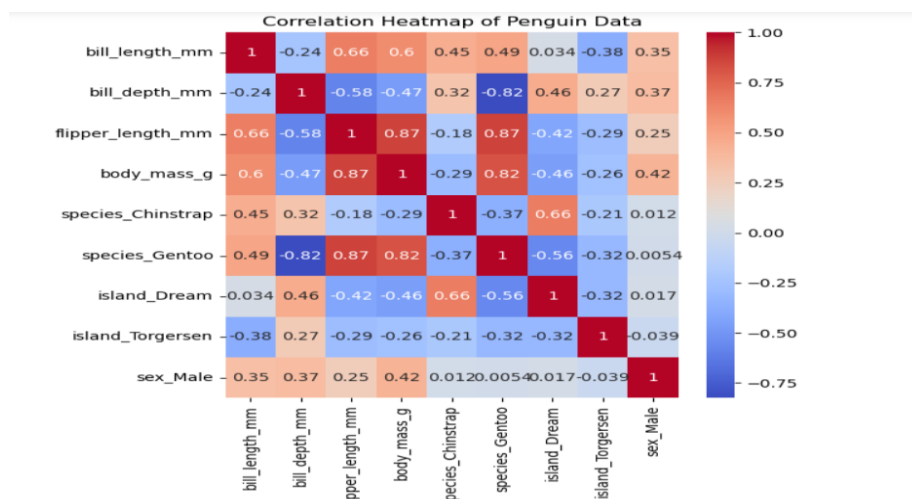
HEATMAP

Heatmaps uses correlation matrix and visualizes data. The datapoints where the higher values get brighter colors and lower values get darker colors.

Code snippet:

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3 import pandas as pd
4
5 penguins_encoded = pd.get_dummies(df, columns=['species','island','sex'], drop_first=True)
6
7 correlation_matrix = penguins_encoded.corr()
8
9 plt.figure(figsize=(6, 6))
10 sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm")
11 plt.title("Correlation Heatmap of Penguin Data")
12 plt.show()
13
```

Output:



Description:

The heatmap visualizes the correlation between variables using different color intensities. The graph indicates strong and weak relationships among variables, where brighter or darker colors represent higher or lower correlation values respectively.

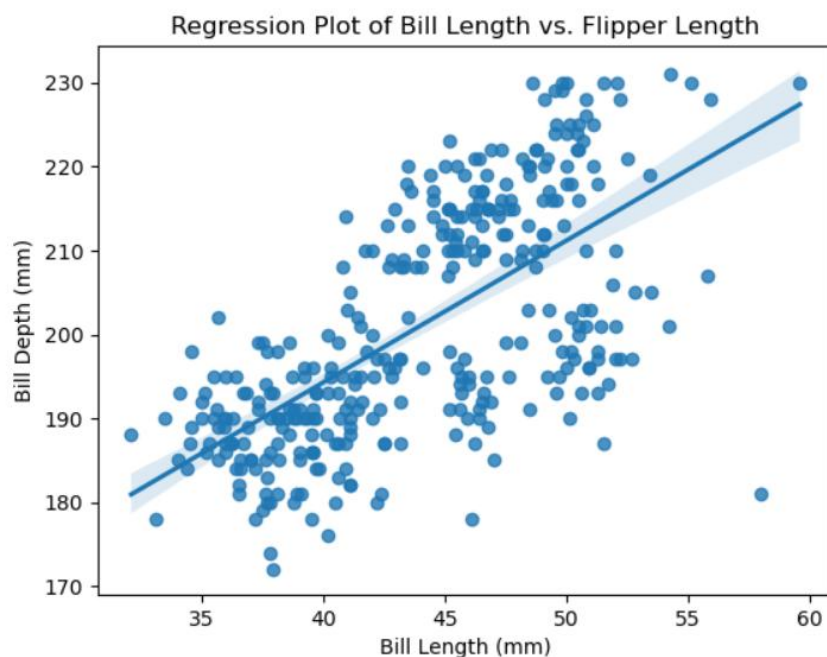
REGPLOT

Regression plots shows the relationship between variables along with the regression line.

Code snippet:

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 sns.regplot(x="bill_length_mm", y="flipper_length_mm", data=penguins)
5 plt.title("Regression Plot of Bill Length vs. Flipper Length")
6 plt.xlabel("Bill Length (mm)")
7 plt.ylabel("Bill Depth (mm)")
8 plt.show()
9
```

Output:



Description:

The regression plot shows the relationship between total bill and tip amount along with a regression line. The graph indicates a positive relationship, meaning that as the total bill increases, the tip amount also tends to increase. The regression line helps identify the overall trend in the dataset.

COMPARISON OF MATPLOTLIB AND SEABORN

Matplotlib:

- **Core Library:** Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. It provides a low-level interface for creating plots with fine-grained control over every aspect of the figure.
- **Flexibility:** Matplotlib allows users to create any type of plot imaginable, from basic line plots and scatter plots to complex 3D plots and geographical maps.
- **Mature:** Matplotlib has been around for a long time and is a well-established library in the Python ecosystem. Many other libraries, including Seaborn, are built on top of Matplotlib.
- **Customization:** Matplotlib offers extensive customization options, allowing users to customize every aspect of a plot, including colors, line styles, markers, fonts, annotations, and more.
- **Integration:** Matplotlib can be easily integrated with other libraries and frameworks, such as NumPy, Pandas, SciPy, which makes it a versatile choice for data visualization in various contexts.

Advantages of Matplotlib:

- High degree of customization and control over plots.
- Wide range of plot types and styles.
- Strong community support and extensive documentation.
- Well-suited for creating publication-quality figures and graphics.

Seaborn:

- **High-Level Interface:** Seaborn is built on top of Matplotlib and provides a high-level interface for creating attractive and informative statistical graphics. It simplifies the process of creating complex visualizations by providing default aesthetics and convenient functions for common tasks.
- **Statistical Plotting:** Seaborn specializes in statistical plotting and offers a variety of plot types specifically designed for visualizing relationships in data, such as scatter plots, bar plots, box plots, violin plots, pair plots, and more.
- **Aesthetics:** Seaborn comes with built-in themes and color palettes that make it easy to create visually appealing plots with minimal effort. It also provides tools for fine-tuning plot aesthetics, such as controlling the size of plot elements and adjusting the color palette.
- **Integration with Pandas:** Seaborn seamlessly integrates with Pandas dataframes, allowing users to easily plot data stored in Pandas data structures without the need for extensive data manipulation.

Advantages of Seaborn:

- Simplified syntax and high-level functions for creating complex plots.
- Built-in support for statistical plotting and data exploration.
- Attractive default aesthetics and color palettes.
- Seamless integration with Pandas dataframes for data visualization.
- Ideal for exploratory data analysis and quick visualization of relationships in data.

Overall, the choice between Matplotlib and Seaborn depends on the specific requirements of data visualization task. Matplotlib is preferred for its flexibility and fine-grained control over plots, while Seaborn is preferred for its ease of use, statistical plotting capabilities, and attractive default aesthetics. Many users combine both libraries, leveraging the strengths of each as needed.